

Github link: <https://github.com/Oz-Asher/Second-Task-in-Neural-Networks/tree/main>

Part 1: CNN Debugging

After training the model with the original CNN architecture we evaluated it using a few basic metrics we learned about: Training and test losses, and training and test accuracies. It was clear that the model wasn't performing very well - test and training accuracies were in the low 60s as can be seen in figure 1 in the actual values and in figure 2 in the plotted line graphs. In order to improve the model we implemented two techniques: Batch normalization and changing the activation function. We ruled out the option of adding another convolutional layer because the max pooling we implemented would shrink the feature map to a resolution of 1*1 pixel which would effectively destroy all remaining spatial relationships before the data ever reaches the linear classification layers. Noticing the vanishing gradient issue, we added batch-normalization and switched the activation function from sigmoid to ReLU. Instead of squashing the data into decimals between 0 and 1 like the sigmoid, ReLU allows positive values to pass through unchanged. This ensures the gradient remains a healthy, whole number (exactly 1) as it backpropagates, allowing the network to train much faster and reach a higher accuracy ceiling. Figure 3 illustrates that the gradients were significantly higher in the improved version. Overall, model performance was significantly improved. However, an overfitting issue came up, which will be discussed in the next sections of this report.

Part 2: Comparison to MLP

After training both models, the required metrics were used to evaluate and compare their performance. The only differences were in the number of parameters, training speed, and estimated memory usage (see figure 4). Although the MLP model had more parameters, and required significantly more memory, the running time was lower than that of the CNN model. The same hyperparameters were used in both models. In terms of test accuracies, the CNN model was 23% more accurate than the MLP, which was not surprising to us because the CNN architecture allows the network to learn specific spatial resolution relationships and extract certain features of the images. The MLP architecture is less suitable for image recognition tasks as it fails to do so because of the single dimension it squishes the data into. Both models suffer from severe overfitting which will be discussed in the next section. This is clearly visible in figure 2 in the bottom left plot, and in figure 5 in the left plot.

Figures 6 and 7 are visualizations of the first layer filters that the models learned. We used these visualizations to try and interpret what the models are doing. In theory, we expected the MLP filters to be less interpretable to the naked human eye than those of the CNN. Intuitively this makes sense because CNN architecture has similar feature extraction capabilities to those of the human visual systems in our brain. Therefore, we looked for edge or color detecting filters in the visualizations that would resemble v1 neural circuitry. In practice, the learned filters of the CNN were not very interpretable, at least not as the ones we saw in textbook examples of more powerful CNN models like the ResNet which are much deeper than those computed in this

assignment. Similarly, the MLP's learned weights were also uninterpretable. The most noticeable difference in the filters of the two different architectures is the spatial resolution: while the CNN learns the picture using a 3*3 kernel, the MLP looks at the entire 32*32 image. This is the main difference from which different strengths and weaknesses arise for the two distinct types of ML algorithms: CNNs are more interpretable and preserve spatial resolution by using localized, sliding filters to extract geometric features, making them ideal for unstructured, grid-like data such as images. Conversely, MLPs function as uninterpretable black boxes because they flatten data into one-dimensional vectors, destroying all spatial relationships to form rigid global templates of the inputs. Because of this structural blindness, MLPs are poorly suited for visual tasks but excel at processing structured, tabular data where features are independent and spatial location is irrelevant.

Part 3: Overfitting Analysis

We computed a larger MLP model with twice as many parameters and retrained it and evaluated it. We can see in figure 8 that the difference between test and training accuracies and test and training losses is a very big difference. Adding all those parameters barely increased test accuracy, and barely reduced the test loss. The overfitting was still severe, and it is clear in figure 9 that it begins as soon as epoch 5.

To experiment with different techniques to deal with the overfitting issue we implemented an early stop, and weight decay. Figure 10 shows how the training process automatically halts with the early stop mechanism, which allows us to save compute power and time by recognizing the issue when it first appears. We used a 'patience' approach allowing for some more epochs to run even after validation loss starts to increase, just to make sure we aren't stopping early for nothing. This technique doesn't solve the overfitting issue, rather it raises awareness to the issue quickly, allowing us to start working on a possible solution earlier.

Conversely, weight decay is a more aggressive approach to dealing with overfitting: it is a deep network solution that perturbs the hidden layers of the system and penalizes weights which are becoming too dominant. In our model, the weight decay did reduce the difference between training and test accuracies and losses. Comparing figure 8 to figure 11 one can see the difference. With weight decay, test (validation) loss was significantly lower, and training accuracy was also lower by almost 10%. The plots in figure 12 show a less severe divergence of the training and test curves, indicating less overfitting. However, the problem still persists and we turned to other possible solutions.

Part 4: Augmentation

To overcome the overfitting hurdle, augmentation was applied to the original CIFAR-10 dataset. Without augmentation, the CNN saw the exact same 50,000 static images epoch after epoch. It had enough parameter capacity to simply memorize the pixel layouts (the background noise, the specific lighting), pushing the training accuracy to a misleadingly high level (overfitting).

By implementing random crops, flips, and rotations, we created a dynamically shifting dataset. The model never saw the exact same image twice. It became mathematically impossible for the network to memorize the dataset. The augmentation effectively destroyed the "shortcuts" the

model was using, forcing the training accuracy to drop down and match the model's true generalization ability (the test accuracy). Figure 15 shows the massive change of the learning curves from the regular improved CNN to the augmented version. Figure 14 shows some of the augmented samples: top left is a zoomed/resized bird image, the next image is a rotated image of a bird. The other samples in the figure have all been augmented with rotations, zoom\resizing, horizontal flips, and random crops. Finally, confusion matrices of the test sets were generated to get a better sense of where the model is struggling. When comparing the matrices at first it appears that the unaugmented CNN performed better. Most of the values in the diagonal are larger before the augmentation. However, it is important to keep in mind that a confusion matrix is blind to overfitting when it is used for the test set alone, without another matrix for the training set to compare to. It is true that the test accuracy was slightly higher before the augmentation, but this does not mean the CNN is better overall. The generalizing capabilities of the augmented CNN are better, and the overall performance of the neural network in terms of application to the real world is better as it did not overfit and memorize the dataset. Lastly, notice that the highest number of errors with or without augmentation are between the cat and dog classes (3 and 5).

Part 5: Creativity Task

Finally a hybrid CNN-MLP model architecture was created. The model was evaluated and compared to the augmented CNN in terms of train and test accuracies and losses, and a confusion matrix was generated to explore the mistakes the network made in in the test set. To deal with the overfitting we used augmentation, and implemented an early stop mechanism as well. Figure 16 shows the entire training process of the hybrid model. We chose to create a CNN feature extractor in the early layers of the models architecture and an MLP classifier in the last layers. Figure 17 shows the model report. Compared to the MLP and CNN models from earlier we can see that the values are right in the middle (memory usage, number of parameters). Average running time of an epoch was longer, and there were 25 epochs in the training process. In terms of performance there were no significant differences when compared to the augmented version of the CNN. The learning curves are displayed in figure 18. In order to make sure we have experimented with all possible variables we decided to change an important hyper parameter just to investigate the effects on the training and performance. We ran the hybrid model again after increasing the hyperparameter from the standard 0.001 to 0.01 and also decreasing it to 0.0001. Model performance was worse with the larger and smaller learning rates. The learning curves are displayed in figure 19. Figure 20 is a schematic drawing of the hybrid model. Finally, we visualized the weights of the MLP classifier layer to see what happens in terms of interpretability. The weights are displayed in figure 21. Clearly the weights are uninterpretable to humans, there are no recognizable shapes or prominent features detecting filters.

The overall accuracy reached a bottleneck around 75% because we are using basic model architectures in this assignment. The common mistakes seen in the confusion matrices of augmented CNN and the hybrid model represent the main flaw of this model's architecture: A shallow 3-layer CNN successfully learned to detect "animals with four legs and fur" (which is why it doesn't confuse a dog with an airplane), but it physically lacks the depth to learn the

nuanced facial features required to tell a dog and a cat apart at a 32x32 resolution. The converging learning curves prove that the model is genuinely learning generalizable features rather than regurgitating memorized noise. The fact that the test accuracy stagnated at ~75% is empirical proof that we successfully solved the overfitting problem and subsequently discovered the absolute performance ceiling of this specific architectural design. In order to create a better model like the ResNet, a deeper network is required, with many more layers and skip-connections, much more compute and a longer running time, larger samples and higher resolution images, attention mechanisms, and more modern techniques that were developed in the world of ML in recent decades.

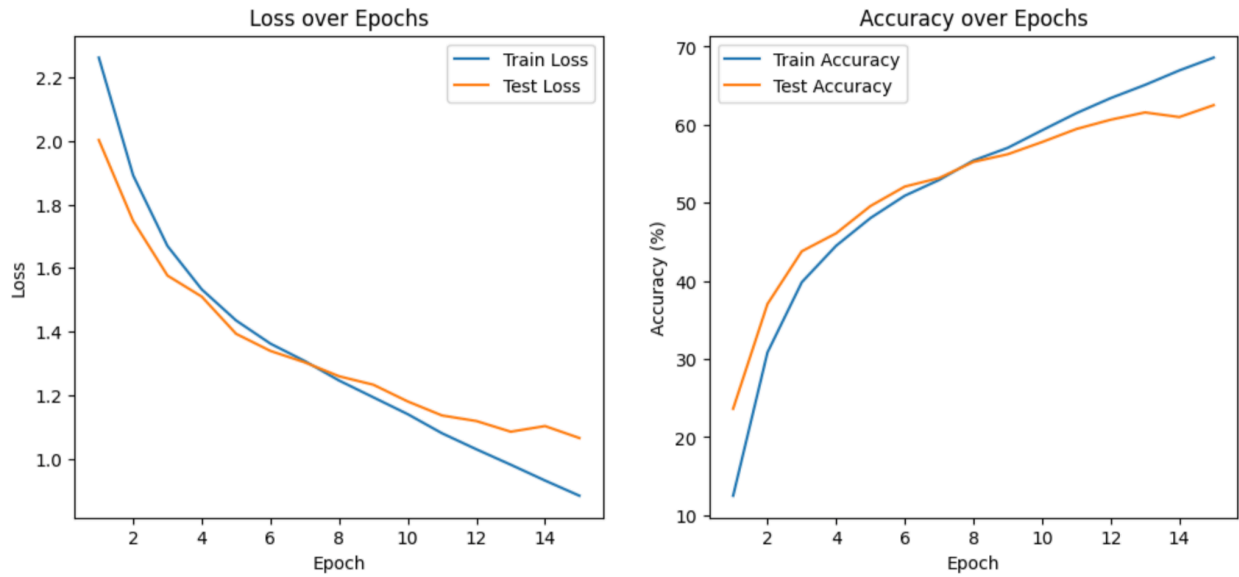
Part 1 Figures:

```
Model before improvement:
Epoch 1/15 | Time 161.67 sec
Train Loss: 2.2624 | Train Accuracy: 12.51%
Test Loss: 2.0030 | Test Accuracy: 23.65%
-----
Epoch 2/15 | Time 164.07 sec
Train Loss: 1.8918 | Train Accuracy: 30.85%
Test Loss: 1.7483 | Test Accuracy: 37.05%
-----
Epoch 3/15 | Time 165.96 sec
Train Loss: 1.6695 | Train Accuracy: 39.84%
Test Loss: 1.5764 | Test Accuracy: 43.80%
-----
Epoch 4/15 | Time 166.26 sec
Train Loss: 1.5328 | Train Accuracy: 44.50%
Test Loss: 1.5101 | Test Accuracy: 46.10%
-----
Epoch 5/15 | Time 166.10 sec
Train Loss: 1.4351 | Train Accuracy: 48.05%
Test Loss: 1.3931 | Test Accuracy: 49.59%
-----
Epoch 6/15 | Time 166.99 sec
Train Loss: 1.3622 | Train Accuracy: 50.91%
Test Loss: 1.3393 | Test Accuracy: 52.08%
-----
Epoch 7/15 | Time 167.66 sec
Train Loss: 1.3074 | Train Accuracy: 52.93%
Test Loss: 1.3034 | Test Accuracy: 53.17%
-----
Epoch 8/15 | Time 167.76 sec
Train Loss: 1.2461 | Train Accuracy: 55.42%
Test Loss: 1.2595 | Test Accuracy: 55.24%
-----
Epoch 9/15 | Time 167.76 sec
Train Loss: 1.1932 | Train Accuracy: 57.04%
Test Loss: 1.2328 | Test Accuracy: 56.22%
-----
Epoch 10/15 | Time 166.93 sec
Train Loss: 1.1403 | Train Accuracy: 59.28%
Test Loss: 1.1803 | Test Accuracy: 57.78%
-----
Epoch 11/15 | Time 166.95 sec
Train Loss: 1.0801 | Train Accuracy: 61.48%
Test Loss: 1.1361 | Test Accuracy: 59.45%
-----
Epoch 12/15 | Time 167.45 sec
Train Loss: 1.0297 | Train Accuracy: 63.41%
Test Loss: 1.1187 | Test Accuracy: 60.64%
-----
Epoch 13/15 | Time 168.15 sec
Train Loss: 0.9812 | Train Accuracy: 65.09%
Test Loss: 1.0851 | Test Accuracy: 61.57%
-----
Epoch 14/15 | Time 168.17 sec
Train Loss: 0.9314 | Train Accuracy: 66.95%
Test Loss: 1.1028 | Test Accuracy: 60.98%
-----
Epoch 15/15 | Time 168.51 sec
Train Loss: 0.8836 | Train Accuracy: 68.59%
Test Loss: 1.0652 | Test Accuracy: 62.50%
-----
```

```
Model after improvement:
Epoch 1/15 | Time 165.14 sec
Train Loss: 1.4763 | Train Accuracy: 45.84%
Test Loss: 1.1386 | Test Accuracy: 59.87%
-----
Epoch 2/15 | Time 163.74 sec
Train Loss: 0.9621 | Train Accuracy: 66.01%
Test Loss: 0.9002 | Test Accuracy: 68.08%
-----
Epoch 3/15 | Time 165.76 sec
Train Loss: 0.7377 | Train Accuracy: 74.15%
Test Loss: 0.8193 | Test Accuracy: 71.66%
-----
Epoch 4/15 | Time 164.04 sec
Train Loss: 0.5945 | Train Accuracy: 79.25%
Test Loss: 0.7570 | Test Accuracy: 74.54%
-----
Epoch 5/15 | Time 165.52 sec
Train Loss: 0.4837 | Train Accuracy: 83.09%
Test Loss: 0.7580 | Test Accuracy: 74.26%
-----
Epoch 6/15 | Time 164.30 sec
Train Loss: 0.3900 | Train Accuracy: 86.37%
Test Loss: 0.7717 | Test Accuracy: 75.37%
-----
Epoch 7/15 | Time 165.21 sec
Train Loss: 0.2954 | Train Accuracy: 89.58%
Test Loss: 0.8486 | Test Accuracy: 74.66%
-----
Epoch 8/15 | Time 164.08 sec
Train Loss: 0.2244 | Train Accuracy: 92.05%
Test Loss: 0.8599 | Test Accuracy: 75.78%
-----
Epoch 9/15 | Time 165.06 sec
Train Loss: 0.1818 | Train Accuracy: 93.50%
Test Loss: 0.9437 | Test Accuracy: 75.60%
-----
Epoch 10/15 | Time 164.11 sec
Train Loss: 0.1472 | Train Accuracy: 94.78%
Test Loss: 1.0786 | Test Accuracy: 74.85%
-----
Epoch 11/15 | Time 164.90 sec
Train Loss: 0.1246 | Train Accuracy: 95.66%
Test Loss: 1.2064 | Test Accuracy: 75.15%
-----
Epoch 12/15 | Time 163.99 sec
Train Loss: 0.1129 | Train Accuracy: 96.04%
Test Loss: 1.1681 | Test Accuracy: 75.33%
-----
Epoch 13/15 | Time 165.52 sec
Train Loss: 0.1088 | Train Accuracy: 96.25%
Test Loss: 1.2528 | Test Accuracy: 74.34%
-----
Epoch 14/15 | Time 163.55 sec
Train Loss: 0.0918 | Train Accuracy: 96.91%
Test Loss: 1.3512 | Test Accuracy: 74.40%
-----
Epoch 15/15 | Time 165.23 sec
Train Loss: 0.0800 | Train Accuracy: 97.21%
Test Loss: 1.2760 | Test Accuracy: 75.36%
-----
```

Figure 1: Training and test losses and accuracies over epochs. On the left is the initial CNN and on the right is the CNN after the improvement.

Initial CNN:



CNN after improvements:

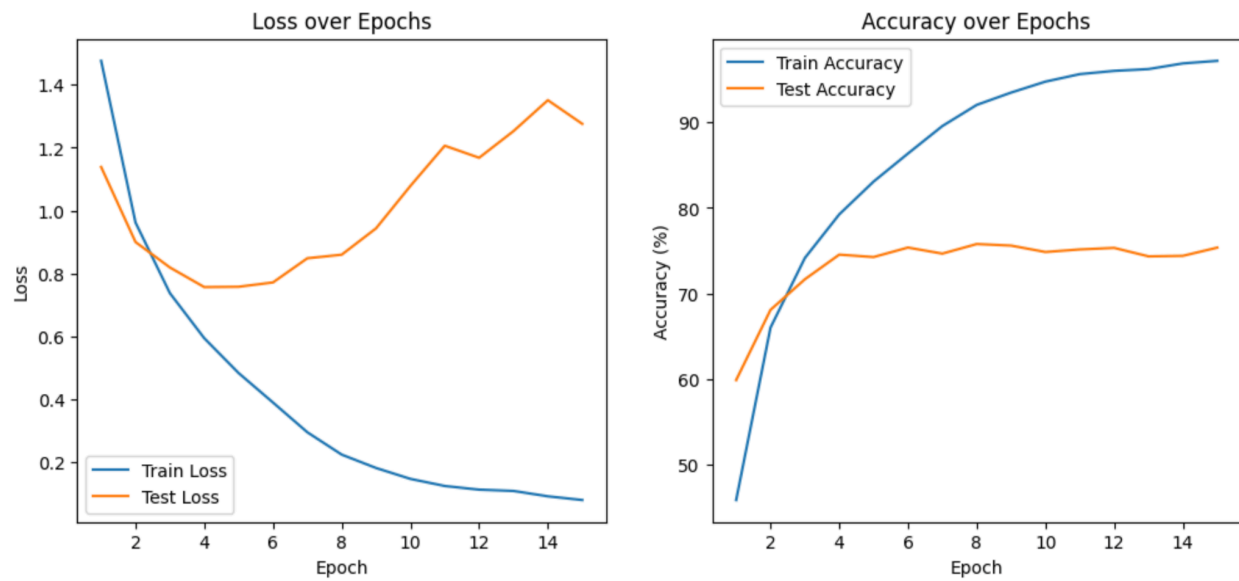
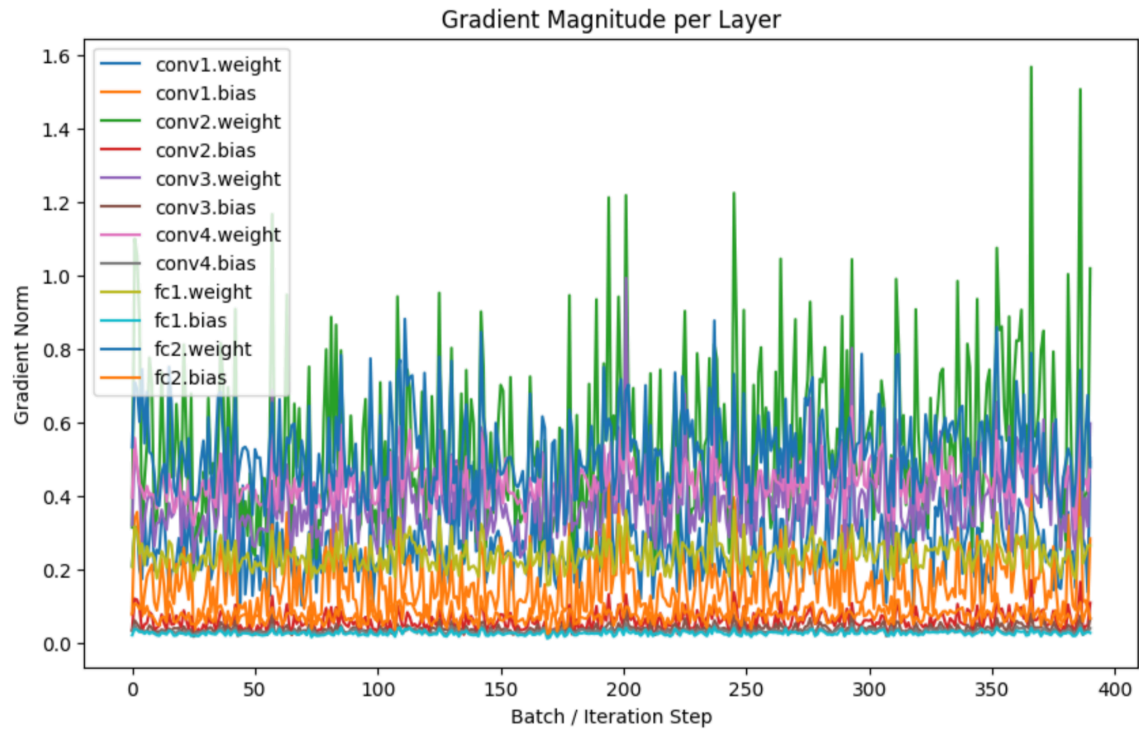


Figure 2: Loss and Accuracy over epochs for the initial CNN on the top and for the CNN after the improvements at the bottom.

Before:



After:

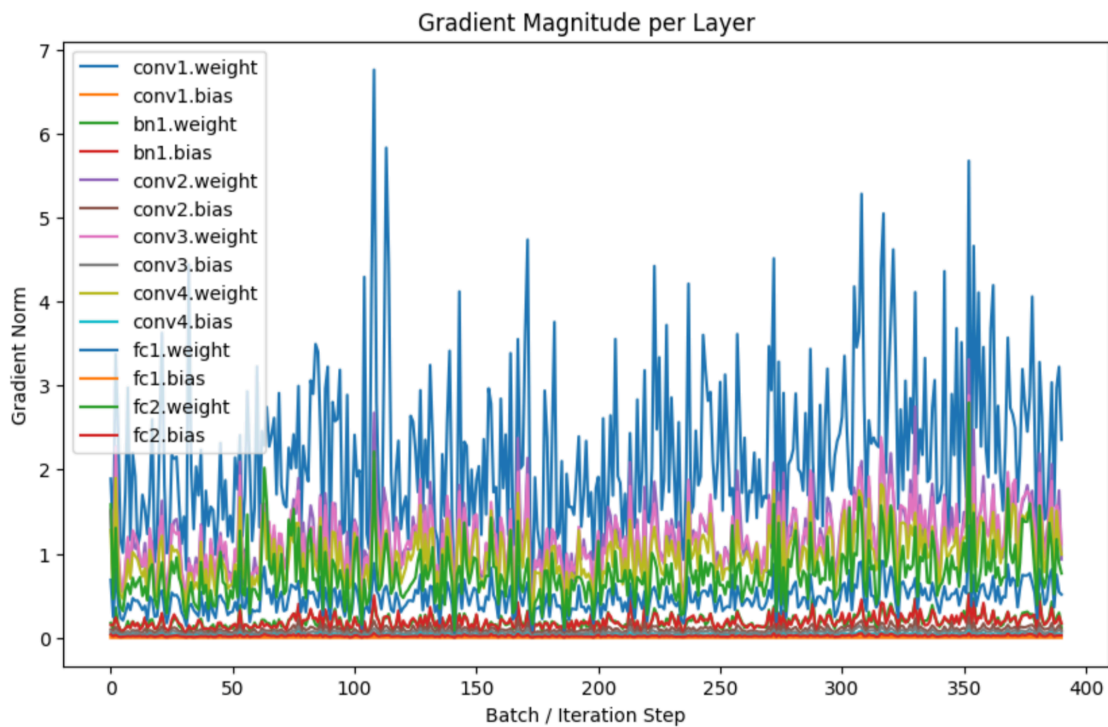


Figure 3: Gradient magnitude per layer for initial CNN before improvement on the top and after the improvement at the bottom.

Part 2 Figures:

Epoch 15/15 Time 15.21 sec Train Loss: 0.0903 Train Accuracy: 96.85% Test Loss: 1.3165 Test Accuracy: 75.54%	Epoch 15/15 Time 12.60 sec Train Loss: 0.4402 Train Accuracy: 84.29% Test Loss: 2.0402 Test Accuracy: 52.54%
=====	=====
MODEL REPORT	MODEL REPORT
=====	=====
Total parameters: 653,450 Trainable parameters: 653,450	Total parameters: 1,707,274 Trainable parameters: 1,707,274
Hyperparameters: Optimizer: Adam Learning rate: 0.001 Batch size: 128	Hyperparameters: Optimizer: Adam Learning rate: 0.001 Batch size: 128
Training speed: 15.97 seconds per epoch on average	Training speed: 12.46 seconds per epoch on average
Estimated memory usage: Parameters memory: 2.49 MB Gradients memory: 2.49 MB Optimizer states memory: 4.99 MB Total estimated: 9.97 MB	Estimated memory usage: Parameters memory: 6.51 MB Gradients memory: 6.51 MB Optimizer states memory: 13.03 MB Total estimated: 26.05 MB
=====	=====

Figure 4: Model reports comparing the improved CNN model (left) to the MLP model (right)

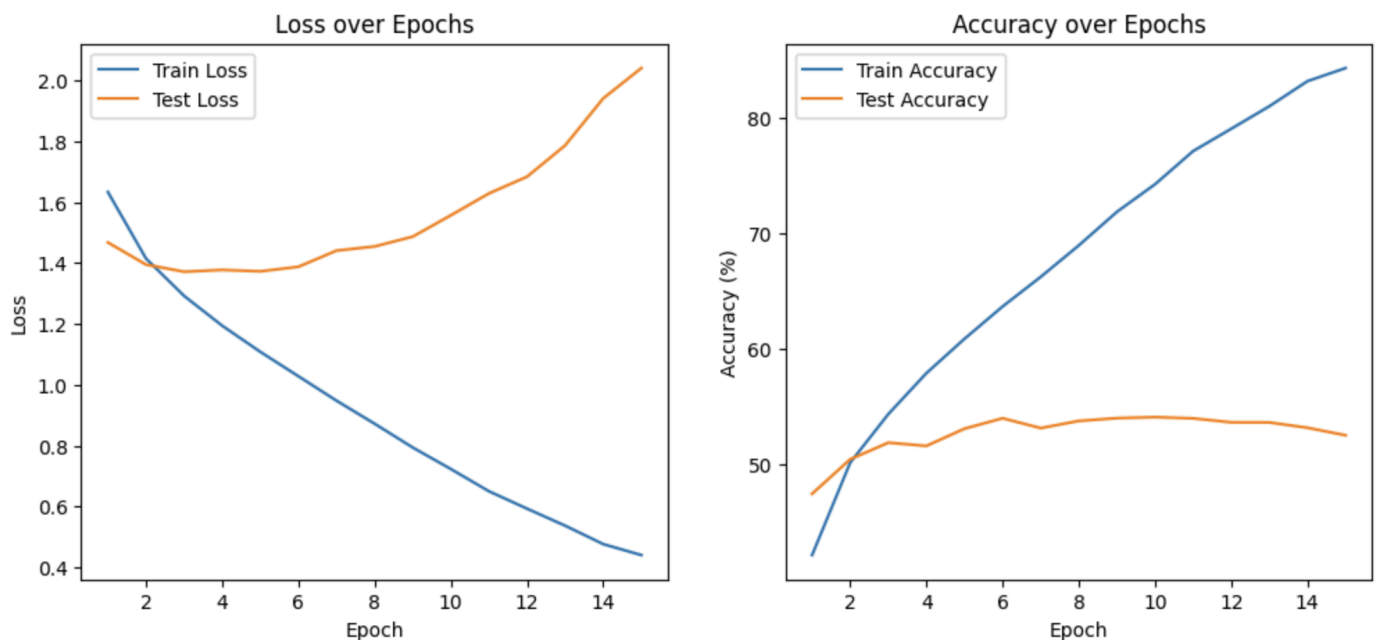


Figure 5: MLP model loss over epochs plot on the left and accuracy over epochs plot on the right.

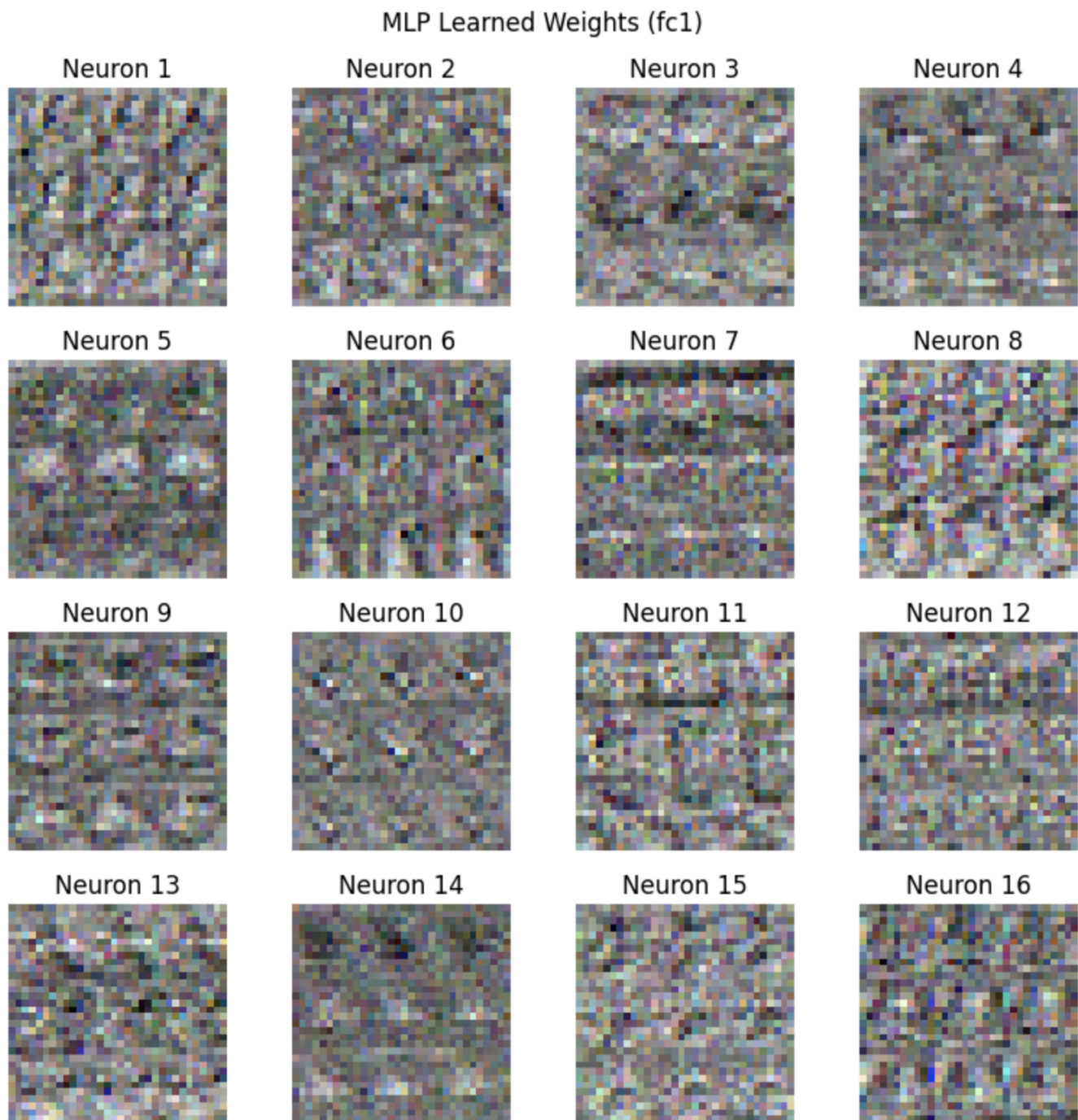


Figure 6: Visualization of MLP first layer filters

CNN Learned Filters (conv1)

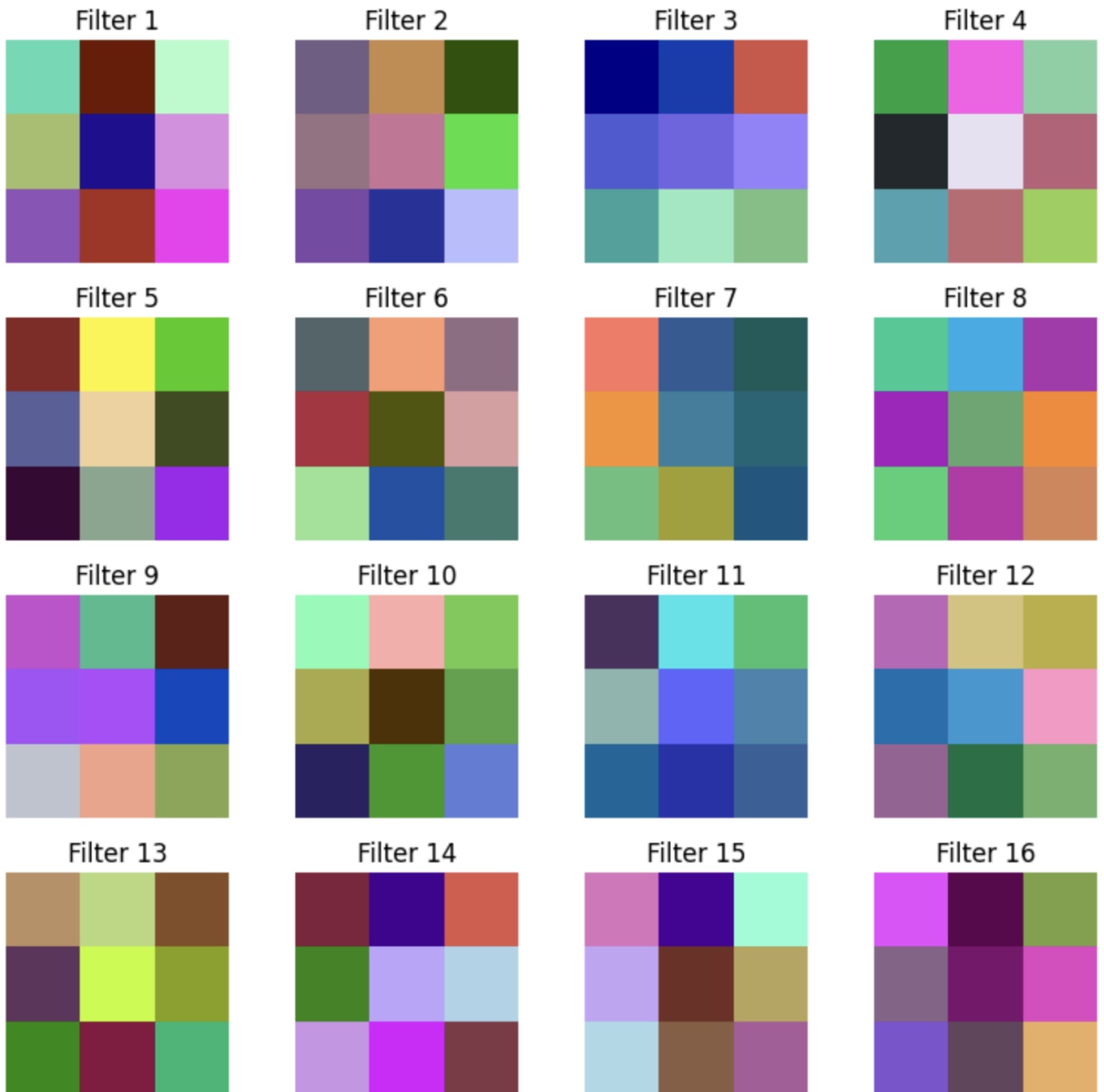


Figure 7: Visualization of CNN first layer filters

Part 3 Figures:

```
Epoch 15/15 | Time 13.24 sec  
Train Loss: 0.3898 | Train Accuracy: 86.34%  
Test Loss: 2.2187 | Test Accuracy: 53.99%
```

Figure 8: Overfitted larger MLP model. Values of loss and accuracy in training compared to the test imply overfitting

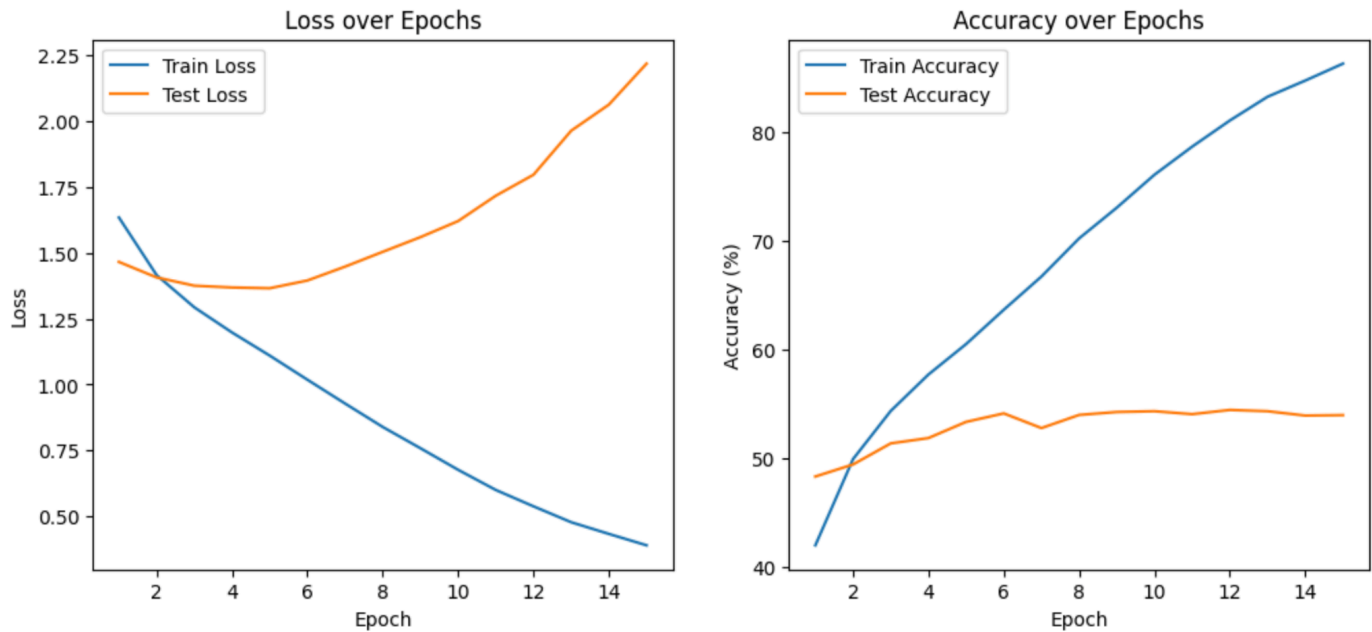


Figure 9: Loss and accuracy over epochs for the overfitted MLP model

```
Adding early stop (Patience = 3 epochs)  
Epoch 1/15 | Time 12.85 sec  
Train Loss: 1.6322 | Train Accuracy: 41.98%  
Test Loss: 1.4803 | Test Accuracy: 47.44%  
-----  
Epoch 2/15 | Time 13.29 sec  
Train Loss: 1.4152 | Train Accuracy: 49.80%  
Test Loss: 1.3907 | Test Accuracy: 50.91%  
-----  
Epoch 3/15 | Time 13.59 sec  
Train Loss: 1.2932 | Train Accuracy: 54.42%  
Test Loss: 1.3648 | Test Accuracy: 52.22%  
-----  
Epoch 4/15 | Time 13.02 sec  
Train Loss: 1.2010 | Train Accuracy: 57.58%  
Test Loss: 1.3600 | Test Accuracy: 52.53%  
-----  
Epoch 5/15 | Time 12.87 sec  
Train Loss: 1.1092 | Train Accuracy: 60.55%  
Test Loss: 1.3787 | Test Accuracy: 52.80%  
-----  
Epoch 6/15 | Time 12.87 sec  
Train Loss: 1.0184 | Train Accuracy: 63.81%  
Test Loss: 1.3913 | Test Accuracy: 54.16%  
-----  
Epoch 7/15 | Time 13.11 sec  
Train Loss: 0.9236 | Train Accuracy: 67.00%  
Test Loss: 1.4480 | Test Accuracy: 53.14%  
-----  
🔴 EARLY STOPPING TRIGGERED at Epoch 7!  
Test loss has not improved for 3 epochs.
```

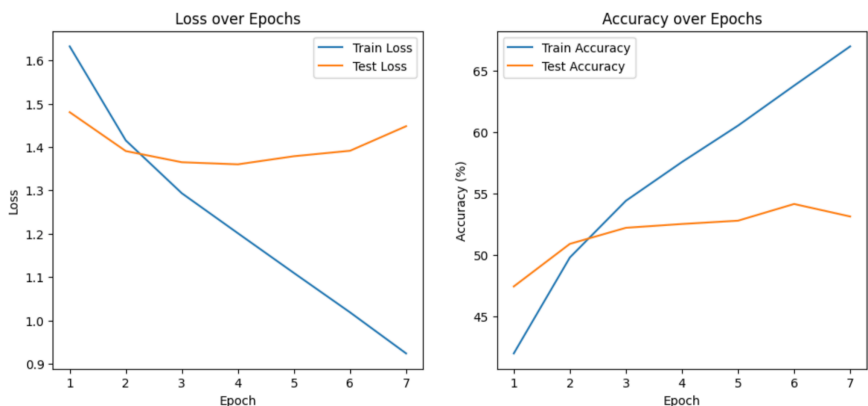


Figure 10: Overfitted MLP evaluation after implementing early stop. The training process was seized automatically at the end of epoch 7


```
Epoch 15/15 | Time 13.04 sec
Train Loss: 0.6543 | Train Accuracy: 76.92%
Test Loss: 1.6439 | Test Accuracy: 53.77%
```

Figure 11: Overfitted MLP evaluation with weight decay. Train and test loss and accuracy after implementing weight decay

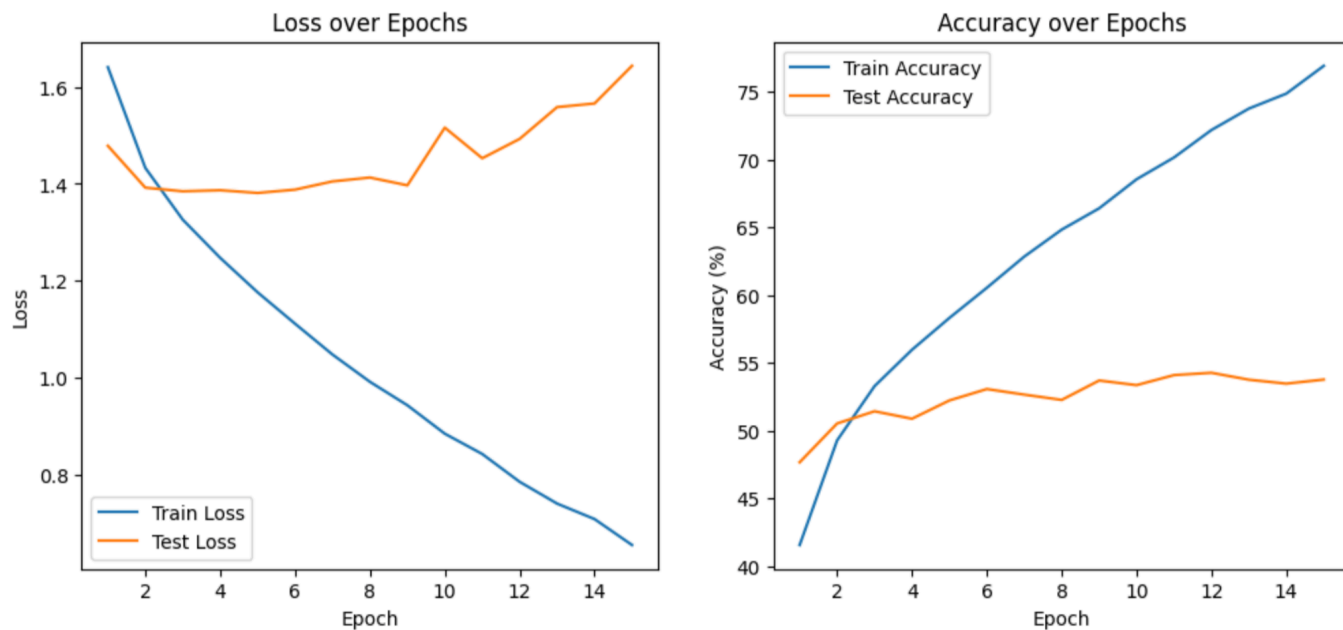


Figure 12: Overfitted MLP training and test loss and accuracy over epochs with weight decay

Part 4 Figures:

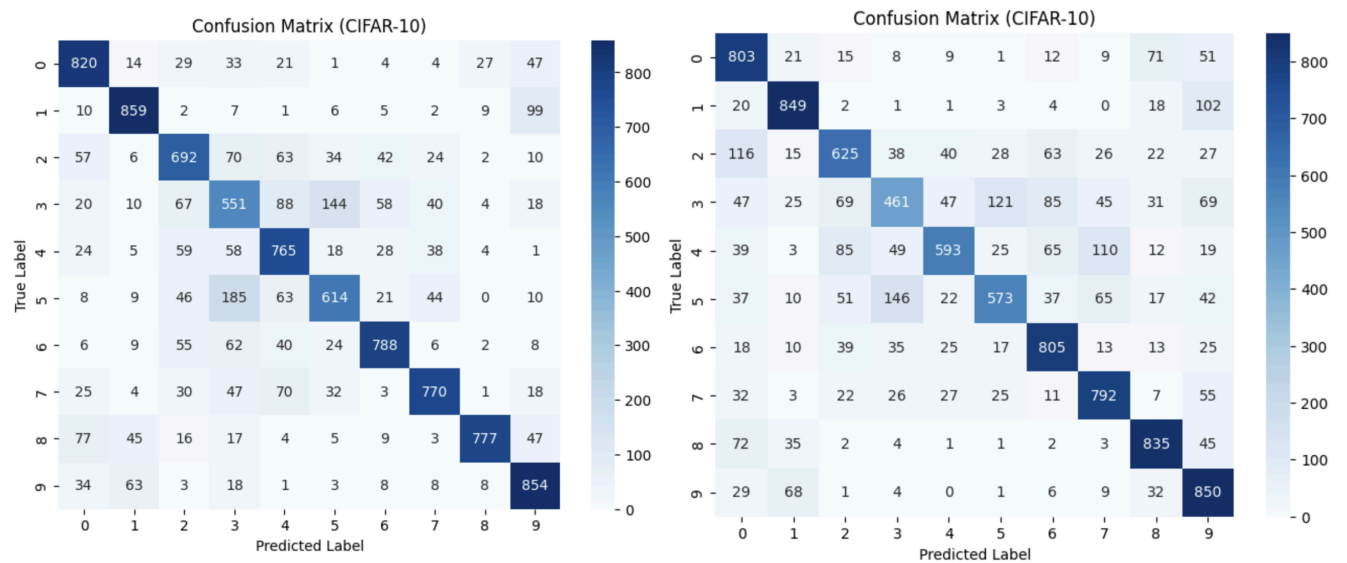


Figure 13: Confusion matrices for the improved CNN model without augmentation on the left, and for the same CNN after implementing augmentation on the right.

0: airplane 1: automobile 2: bird 3: cat 4: deer 5: dog 6: frog 7: horse 8: ship 9: truck

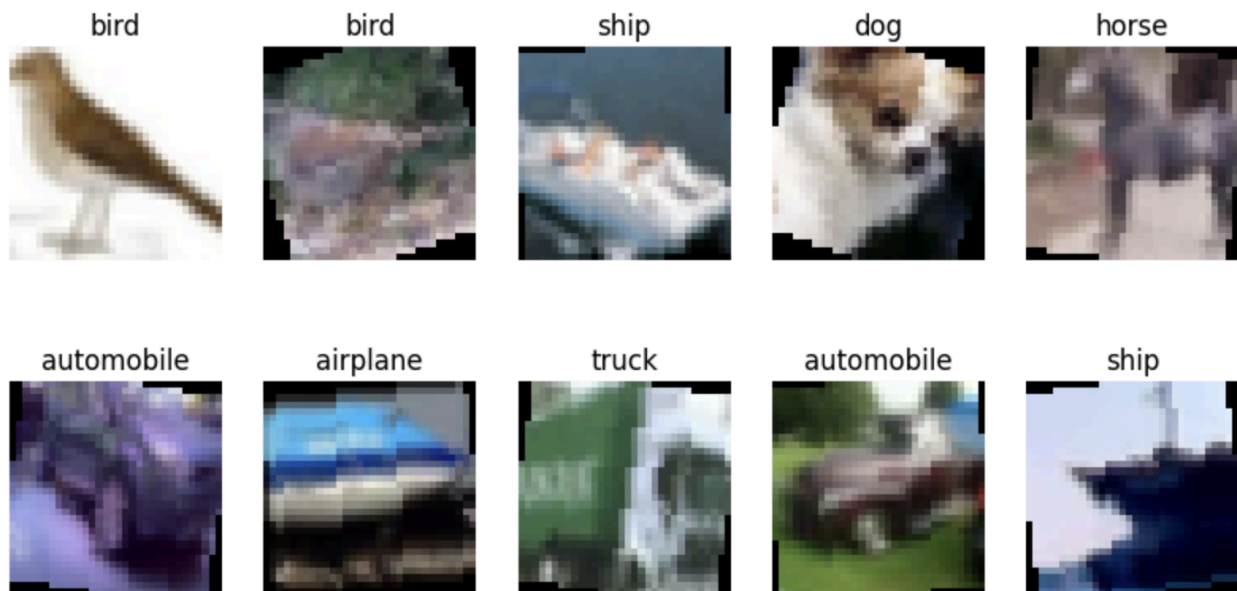
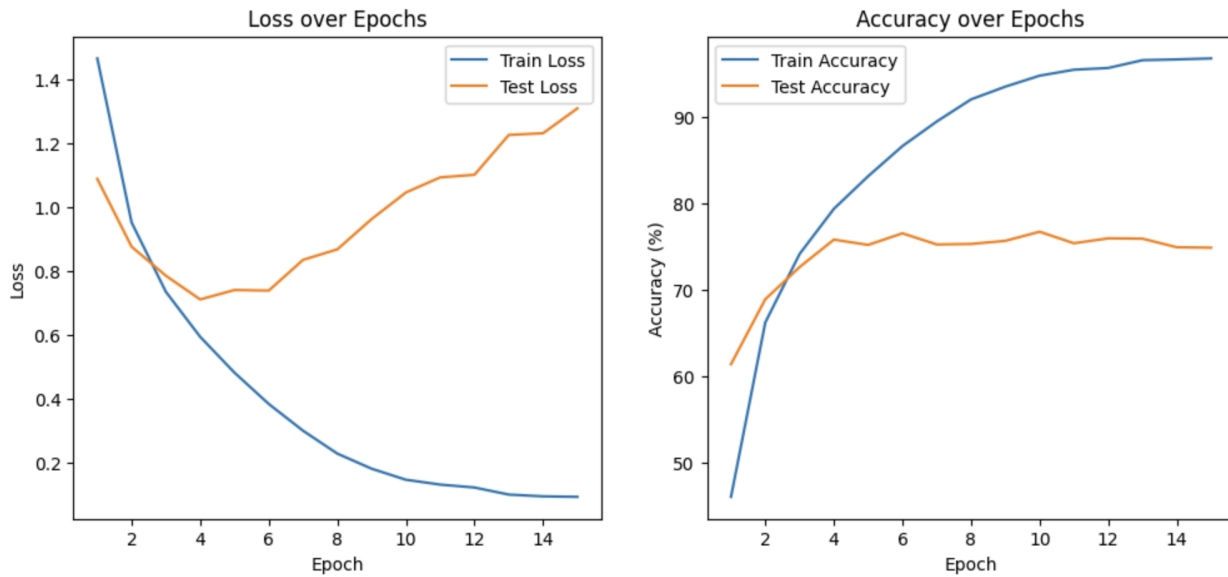


Figure 14: Augmented subset of images from the data set.

Before Augmentation:



After Augmentation:

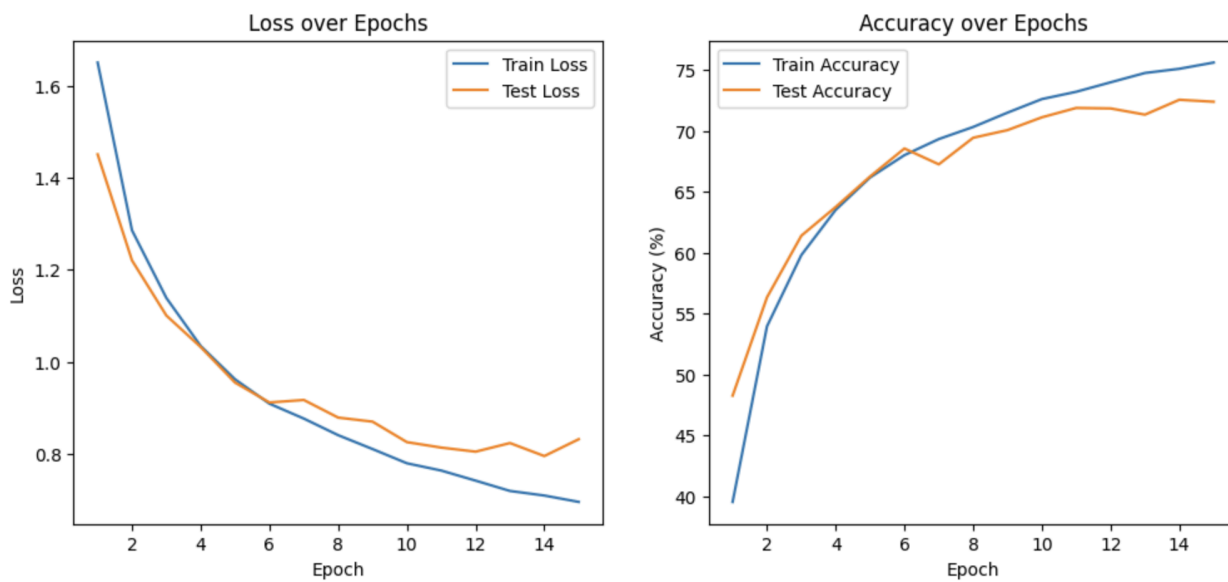


Figure 15: CNN model evaluation before and after augmentation

Part 5 Figures:

```
=====
TASK 5: HYBRID CNN-MLP MODEL (Augmented + Early Stop)
=====
100%|██████████| 170M/170M [00:06<00:00, 27.2MB/s]
Epoch 1/25 | Time 32.49 sec
Train Loss: 1.6758 | Train Accuracy: 37.80%
Test Loss: 1.4647 | Test Accuracy: 46.30%
-----
Epoch 2/25 | Time 31.68 sec
Train Loss: 1.4033 | Train Accuracy: 49.28%
Test Loss: 1.2532 | Test Accuracy: 55.02%
-----
Epoch 3/25 | Time 31.35 sec
Train Loss: 1.2893 | Train Accuracy: 54.19%
Test Loss: 1.1402 | Test Accuracy: 58.91%
-----
Epoch 4/25 | Time 31.14 sec
Train Loss: 1.2119 | Train Accuracy: 57.06%
Test Loss: 1.0765 | Test Accuracy: 61.81%
-----
Epoch 5/25 | Time 31.37 sec
Train Loss: 1.1487 | Train Accuracy: 59.53%
Test Loss: 1.0831 | Test Accuracy: 62.23%
-----
Epoch 6/25 | Time 30.98 sec
Train Loss: 1.1024 | Train Accuracy: 61.52%
Test Loss: 1.0167 | Test Accuracy: 63.74%
-----
Epoch 7/25 | Time 31.41 sec
Train Loss: 1.0567 | Train Accuracy: 63.33%
Test Loss: 0.9914 | Test Accuracy: 65.67%
-----
Epoch 8/25 | Time 31.10 sec
Train Loss: 1.0296 | Train Accuracy: 64.22%
Test Loss: 0.9430 | Test Accuracy: 66.53%
-----
Epoch 9/25 | Time 31.12 sec
Train Loss: 0.9982 | Train Accuracy: 65.66%
Test Loss: 0.9338 | Test Accuracy: 67.21%
-----
Epoch 10/25 | Time 31.03 sec
Train Loss: 0.9745 | Train Accuracy: 66.37%
Test Loss: 0.8951 | Test Accuracy: 68.27%
-----
Epoch 11/25 | Time 31.16 sec
Train Loss: 0.9470 | Train Accuracy: 67.20%
Test Loss: 0.8513 | Test Accuracy: 70.66%
-----
Epoch 12/25 | Time 31.43 sec
Train Loss: 0.9291 | Train Accuracy: 68.00%
Test Loss: 0.8871 | Test Accuracy: 68.88%
-----
Epoch 13/25 | Time 31.36 sec
Train Loss: 0.9106 | Train Accuracy: 68.68%
Test Loss: 0.8911 | Test Accuracy: 68.94%
-----
Epoch 14/25 | Time 31.53 sec
Train Loss: 0.8939 | Train Accuracy: 69.31%
Test Loss: 0.8596 | Test Accuracy: 70.05%
-----
Epoch 15/25 | Time 31.75 sec
Train Loss: 0.8667 | Train Accuracy: 70.27%
Test Loss: 0.8337 | Test Accuracy: 70.81%
-----
Epoch 16/25 | Time 31.23 sec
Train Loss: 0.8593 | Train Accuracy: 70.65%
Test Loss: 0.7886 | Test Accuracy: 72.38%
-----
Epoch 17/25 | Time 31.25 sec
Train Loss: 0.8441 | Train Accuracy: 71.10%
Test Loss: 0.7881 | Test Accuracy: 72.71%
-----
Epoch 18/25 | Time 31.34 sec
Train Loss: 0.8339 | Train Accuracy: 71.73%
Test Loss: 0.8178 | Test Accuracy: 71.42%
-----
Epoch 19/25 | Time 31.33 sec
Train Loss: 0.8145 | Train Accuracy: 72.11%
Test Loss: 0.8343 | Test Accuracy: 71.01%
-----
Epoch 20/25 | Time 31.34 sec
Train Loss: 0.8057 | Train Accuracy: 72.40%
Test Loss: 0.7698 | Test Accuracy: 72.94%
-----
Epoch 21/25 | Time 31.07 sec
Train Loss: 0.7988 | Train Accuracy: 72.76%
Test Loss: 0.7741 | Test Accuracy: 73.11%
-----
Epoch 22/25 | Time 31.11 sec
Train Loss: 0.7849 | Train Accuracy: 73.46%
Test Loss: 0.7863 | Test Accuracy: 72.63%
-----
Epoch 23/25 | Time 31.22 sec
Train Loss: 0.7749 | Train Accuracy: 73.57%
Test Loss: 0.7358 | Test Accuracy: 74.42%
-----
Epoch 24/25 | Time 31.78 sec
Train Loss: 0.7665 | Train Accuracy: 73.91%
Test Loss: 0.7580 | Test Accuracy: 74.22%
-----
Epoch 25/25 | Time 31.28 sec
Train Loss: 0.7610 | Train Accuracy: 74.03%
Test Loss: 0.7292 | Test Accuracy: 74.62%
=====
```

Figure 16: Hybrid CNN-MLP training process

```

=====
MODEL REPORT
=====
Total parameters: 1,209,738
Trainable parameters: 1,209,738

Hyperparameters:
Optimizer: Adam
Learning rate: 0.001
Batch size: 128

Training speed:
31.35 seconds per epoch on average

Estimated memory usage:
Parameters memory: 4.61 MB
Gradients memory: 4.61 MB
Optimizer states memory: 9.23 MB
Total estimated: 18.46 MB
=====

```

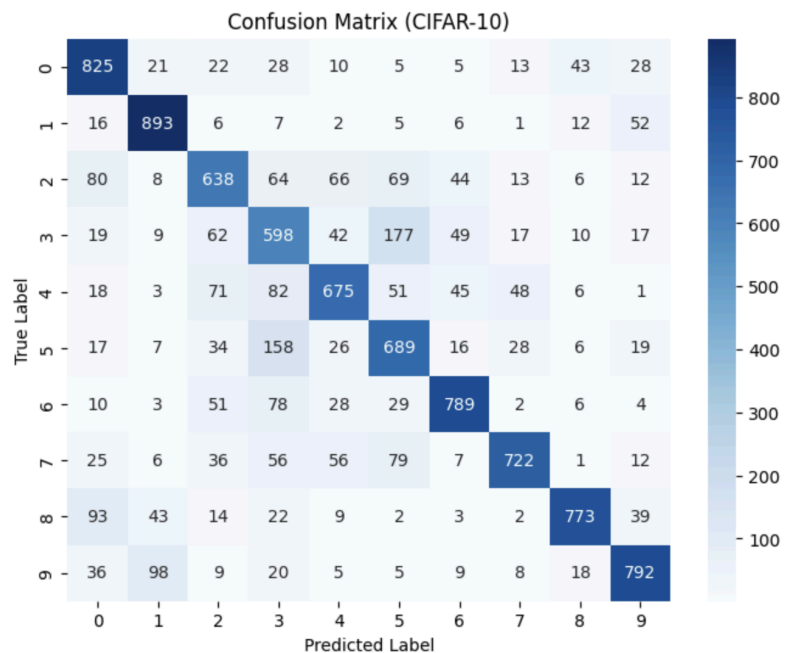


Figure 17: Model report of hybrid CNN-MLP model architecture (left) and confusion matrix (right)

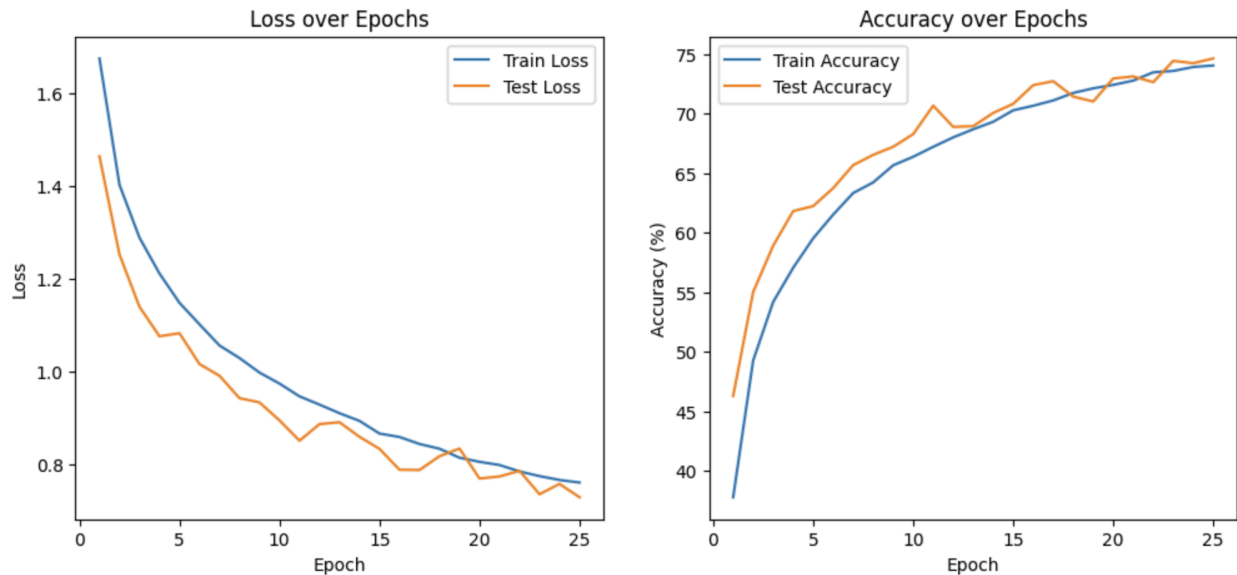
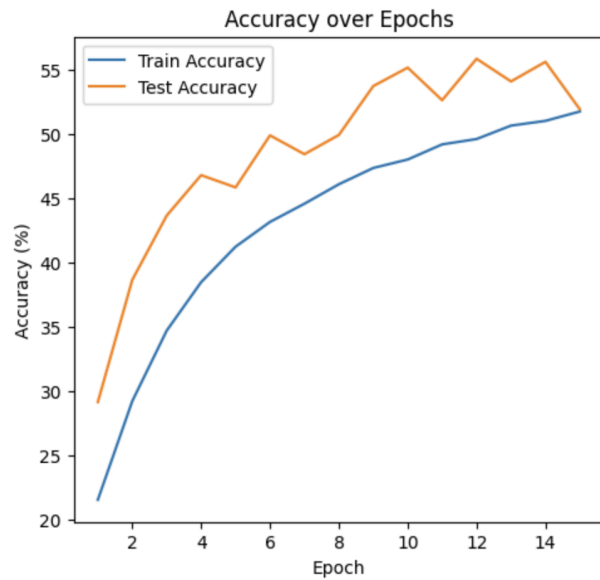
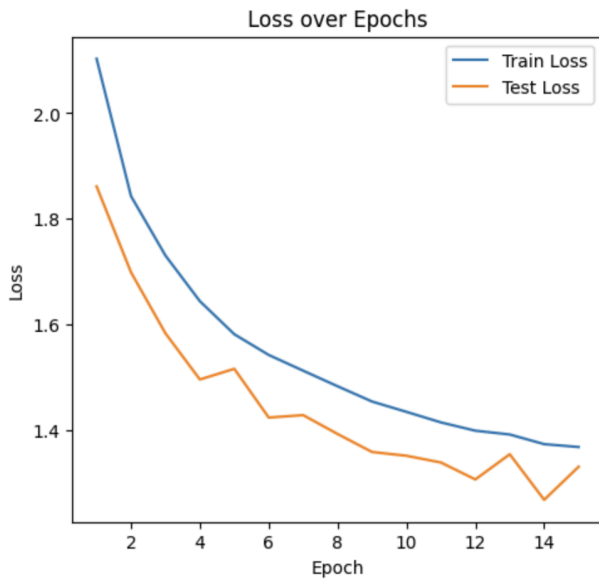


Figure 18: Hybrid CNN-MLP training and test loss and accuracy over epochs

LR=0.01

Epoch 15/15 | Time 43.67 sec
Train Loss: 1.3674 | Train Accuracy: 51.73%
Test Loss: 1.3300 | Test Accuracy: 51.89%



LR=0.0001

Epoch 15/15 | Time 31.75 sec
Train Loss: 1.0320 | Train Accuracy: 63.86%
Test Loss: 0.9691 | Test Accuracy: 65.63%

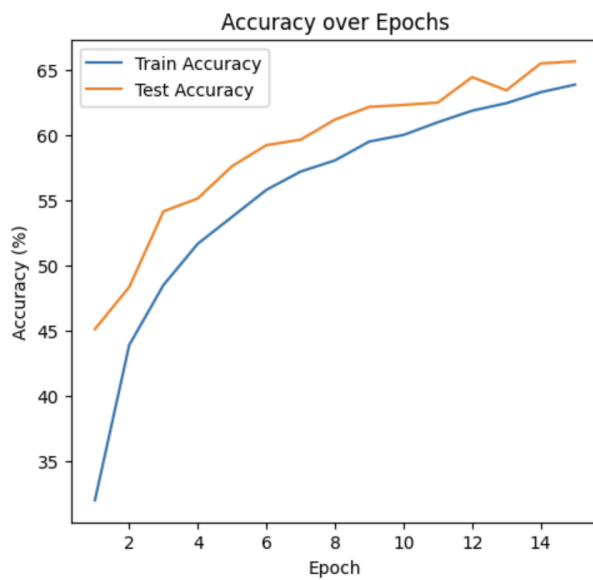
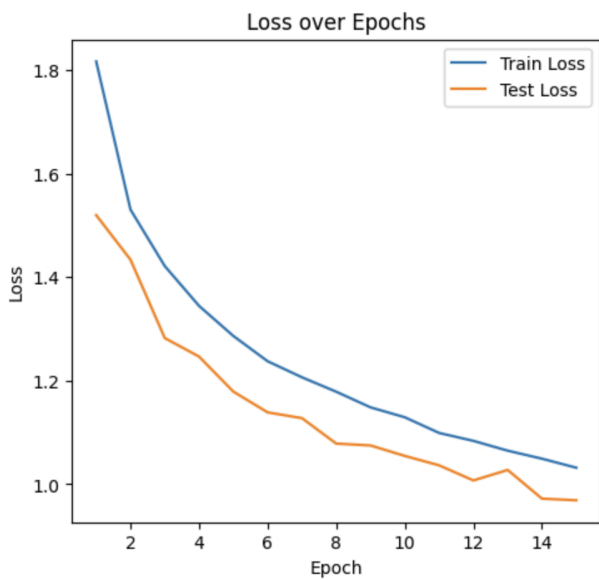


Figure 19: Learning curves after changing the learning rate (LR)

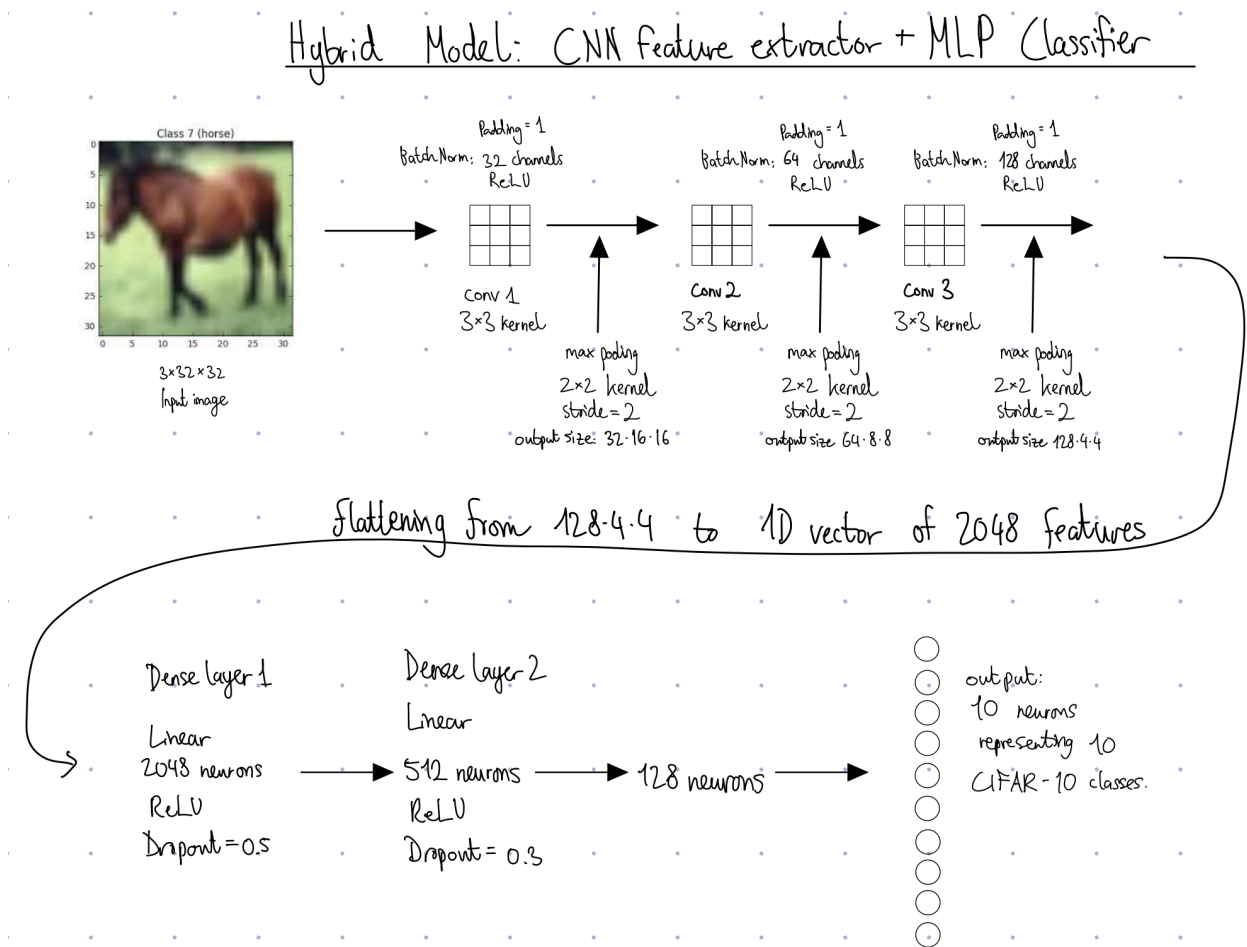


Figure 20: Hybrid model diagram

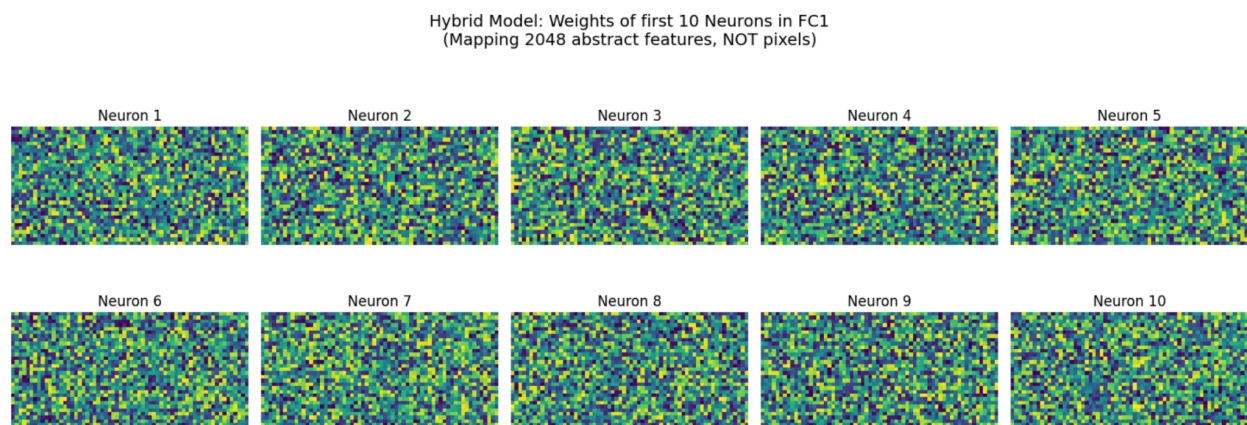


Figure 21: Visualization of FC1 of hybrid model